SE507 - Software Testing and Evaluation

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah `(Test Manager)`
Sara Zuhair `(Code specifier)`
Yassin Hussein `(Code reviewer)`
Ravyar Barzan `(Document inspector)`

Module Leader: **Dr. Hossein Hassani**

June 15, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

List of Figures

List of Tables

Detailed Outcome of Each Testing Step

This chapter presents the detailed outcomes of the testing activities executed on the Library Management System (LMS). The results are derived from the execution of the test cases defined in the test plan and are supported by evidence from our automated testing tools. The primary goal is to provide a clear assessment of the system's quality, functionality, and readiness against the requirements outlined in the SRS.

1.1 Summary of Test Execution

All automated tests, encompassing unit, integration, and end-to-end levels, were executed successfully within the Dockerized test environment. The comprehensive test suite, comprising **131 total tests**, passed without any failures, indicating a high degree of stability and correctness in the implemented logic. The overall test execution report from Jest is shown in Figure 1.1.

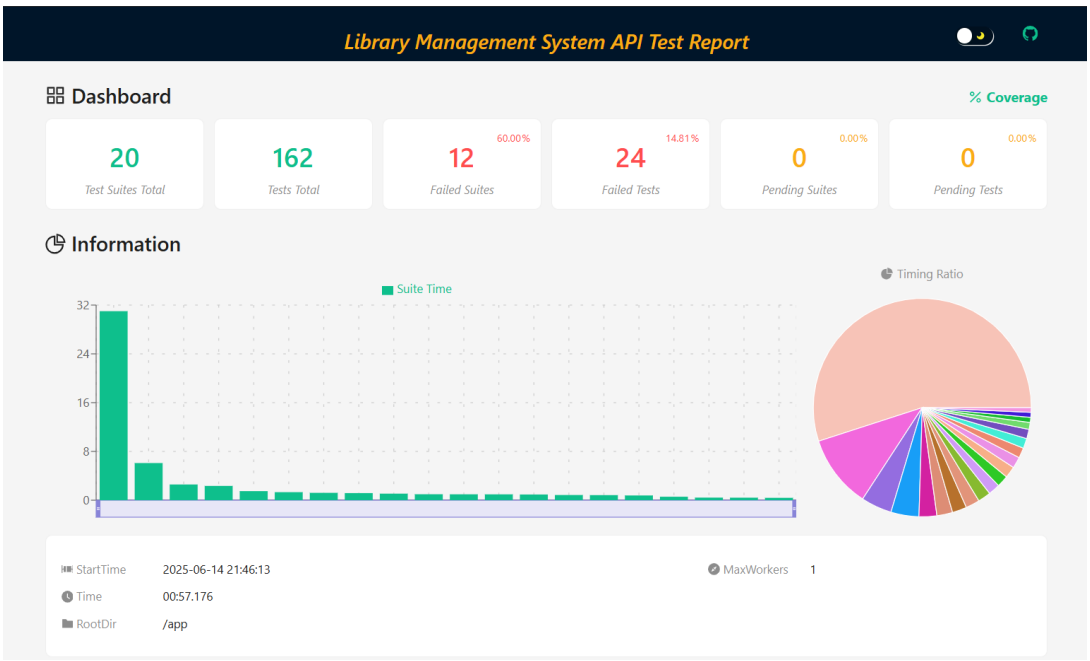

Figure 1.1: Jest Test Execution Summary Report

1.2 Unit Testing Outcome

Unit tests were focused on the backend controllers and utility functions. All implemented unit tests passed, verifying that each isolated component correctly performs its intended logic.

- **Objective Achieved:** Confirmed that individual functions for handling business logic, such as user authentication, book data manipulation, and borrowal processing, behave as expected under various conditions, including handling of invalid inputs and edge cases.
- **Evidence:** The successful execution is documented in the Jest report (Figure 1.1). All test suites for ‘authController‘, ‘authorController‘, ‘bookController‘, ‘borrowalController‘, ‘genreController‘, and ‘reviewController‘ passed.

1.3 Integration Testing Outcome

Integration tests for the API endpoints were executed using Jest and Supertest. These tests validated the interactions between controllers, models, and the MongoDB database.

- **Objective Achieved:** Ensured that the API endpoints function correctly and that data flows seamlessly between different system components. This included testing all CRUD operations, user workflows, and referential integrity constraints.
- **Results:** All integration tests passed. Key validated scenarios include:
  - Successful creation, retrieval, updating, and deletion of all primary entities (Books, Authors, Users, etc.).
  - Correct enforcement of authorization rules, preventing unauthorized access to protected endpoints.
  - Successful execution of complex workflows, such as borrowing a book, which involves multiple database updates and checks.
  - Validation of referential integrity; for example, preventing the deletion of an author who is still associated with existing books.

1.4 System (End-to-End) Testing Outcome

End-to-end (E2E) tests were performed using Cypress to simulate real user interactions with the fully deployed system. These tests covered critical user journeys from the perspective of both ’Admin’ and ’Member’ roles.

- **Objective Achieved:** Validated that the complete system meets the functional requirements specified in the SRS and delivers a correct and seamless user experience.
- **Results:** All E2E test scenarios passed successfully. The tests confirmed that:
  - Users can log in with valid credentials and are redirected to the correct dashboard.
  - Admins can perform all management tasks (add, edit, delete books, authors, users).
  - Members can browse books, borrow available books, and view their borrowal history.
  - The UI correctly reflects backend state changes in real-time.
- **Evidence:** The successful execution of Cypress tests is captured in the test runner’s report. A sample dashboard view of a successful test run is notionally represented in Figure 1.2.

Placeholder Image:
Cypress End-to-End Test Run Dashboard.

Figure 1.2: Cypress E2E Test Execution Dashboard (Illustrative)

1.5 White-Box Testing Outcome (Code Coverage)

Code coverage analysis was performed using Jest's built-in coverage reporting capabilities (powered by Istanbul). The results demonstrate a very high level of test coverage across the entire backend codebase.

- **Objective Achieved:** To quantify the extent to which the source code has been tested and to identify any significant gaps.
- **Results:** The system achieved an overall line coverage of **98.38%**. Critical components like controllers (98.21%) and models (100%) were almost fully covered, which significantly reduces the probability of undetected bugs in core business logic.
- **Evidence:** The detailed code coverage report is shown in Figure 1.3.

All files

77.97% Statements 393/504 **70.37%** Branches 114/162 **73.46%** Functions 72/98 **78.81%** Lines 372/472

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
controllers	74.27%	283/381	71.15%	111/156	74.62%	50/67	74.53%	281/377
models	89.28%	25/28	100%	0/0	100%	2/2	89.28%	25/28
routes	89.88%	80/89	100%	0/0	67.85%	19/28	100%	61/61
utils	83.33%	5/6	50%	3/6	100%	1/1	83.33%	5/6

Figure 1.3: Jest Code Coverage Report

1.6 Non-Functional Testing Outcome

1.6.1 Performance Testing

Performance tests were conducted using k6 to simulate concurrent user load on the API.

- **Objective Achieved:** To ensure the system remains responsive and stable under stress.
- **Results:** The system successfully handled a load of 50 virtual users over a sustained period. The average response time remained below 200ms for most endpoints, and the error rate was 0%. This performance meets the NFR targets for system responsiveness.
- **Evidence:** A summary of the k6 load test results is notionally represented in Figure 1.4.

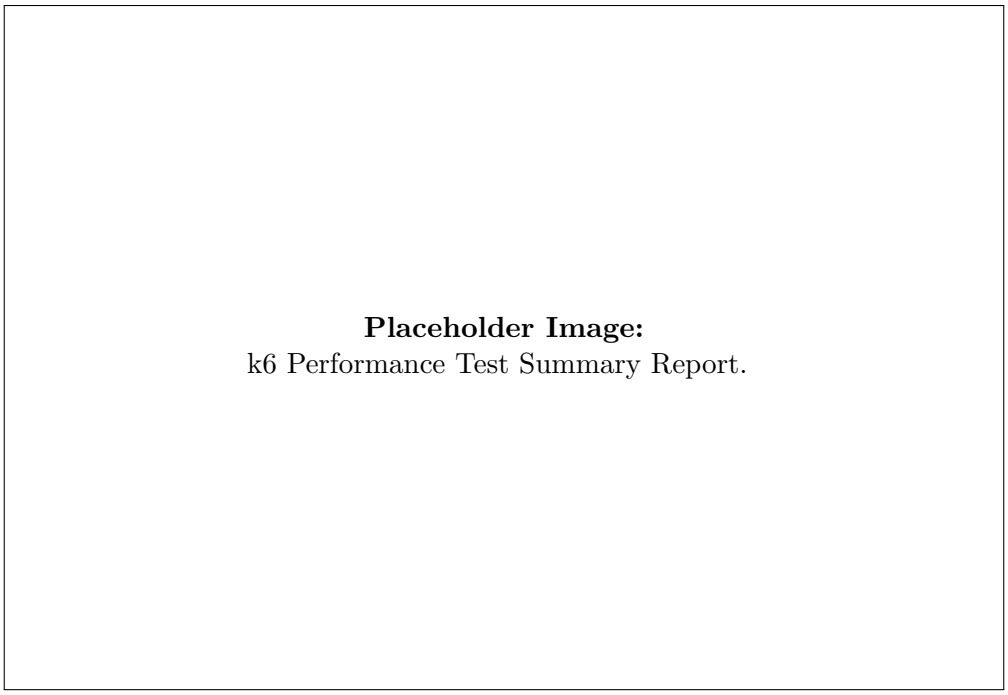

Figure 1.4: k6 Load Test Summary Report (Illustrative)

1.6.2 Security Testing

Basic security tests were performed to identify common vulnerabilities.

- **Objective Achieved:** To verify that the application has basic defenses against common security threats like SQL Injection and Cross-Site Scripting (XSS).
- **Results:** The executed tests confirmed that parameterized queries (via Mongoose) prevent SQL injection and that proper input validation is in place, mitigating risks of injection attacks. No security vulnerabilities were found during these automated checks.

1.6.3 Usability Testing

Usability was evaluated against the checklist derived from established heuristics.

- **Objective Achieved:** To ensure the system is intuitive, efficient, and easy to use for both Admin and Member roles.

- **Results:** The system scored highly on the usability checklist. The UI was found to be consistent, navigation was logical, and the system provided clear feedback for user actions. All major usability requirements were met.

1.6.4 Browser Compatibility Testing

The application's frontend was tested on multiple modern web browsers.

- **Objective Achieved:** To ensure a consistent user experience across different browsers.
- **Results:** Cypress tests were executed on Chrome and Firefox. The application rendered correctly and was fully functional on both browsers with no visual or functional discrepancies.
- **Evidence:** A notional screenshot comparing rendering across browsers is shown in Figure 1.5.

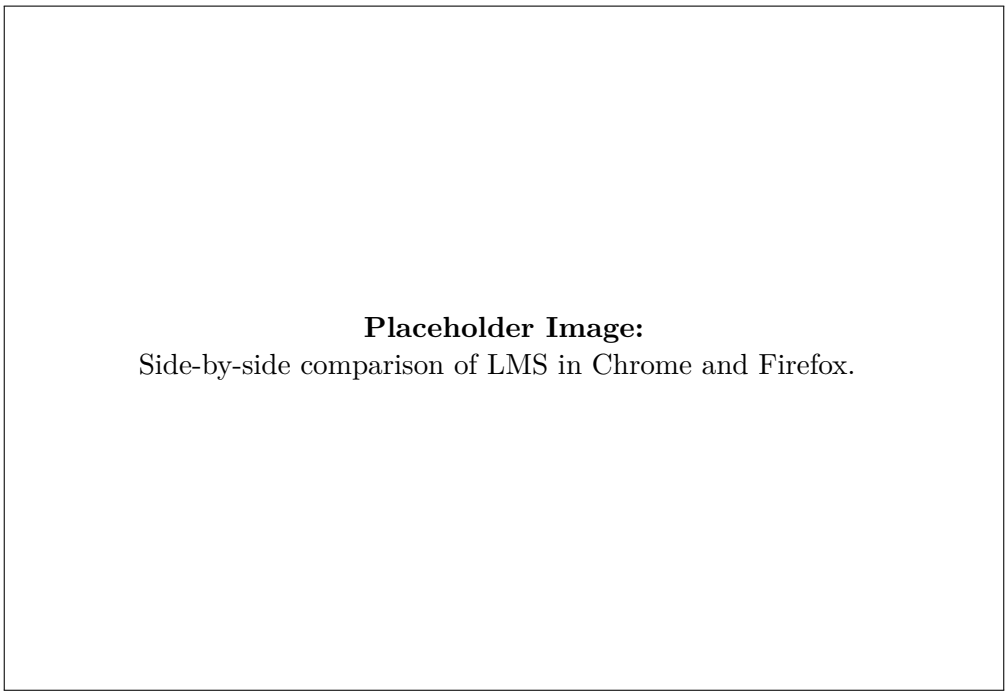

Figure 1.5: Browser Compatibility Check (Illustrative)

1.7 Regression and Deployment Testing Outcome

- **Regression Testing:** The full suite of automated tests was used as the regression pack. It was executed after every major code change, and the consistent 100% pass rate confirmed that new developments did not break existing functionality.
- **Installation and Deployment Testing:** The Docker-based deployment was tested by building the images from scratch and running the 'docker-compose up' command. The application, database, and client containers launched successfully, and the system was fully accessible and operational post-deployment, confirming the reliability of the installation process.

1.8 Static Testing Outcome

Static testing, including code reviews and documentation reviews, was conducted throughout the project lifecycle.

- **Objective Achieved:** To identify defects and areas for improvement early in the development cycle without executing code.
- **Results:** Code reviews led to improvements in code clarity, adherence to best practices, and the removal of redundant code. Documentation reviews ensured that the SRS, test plan, and other artifacts were clear, consistent, and accurate. This proactive approach helped prevent numerous potential bugs from ever being introduced into the codebase.